Solution Tutorial 3

1. **Balancing Chemical Equations**

One useful application of solving systems of linear equations is to balance chemical equations. For simpler equations, this can be done more quickly by hand, however for complicated equations, being able to solve them numerically can be very handy. Consider the following chemical equation:

$$H_3PO_4 + (NH_4)_2MoO_4 + HNO_3 \rightarrow (NH_4)_3PO_4.12MoO_3 + NH_4NO_3 + H_2O$$

To balance the equation, we first put coefficients x_1 to x_6 before each part of the above equation and construct linear equations for each of the elements H, P, O, N, and Mo as follows:
$H : 3x_1 + 8x_2 + 1x_3 = 12x_4 + 4x_5 + 2x_6$
$P : 1x_1 + 0x_2 + 0x_3 = 1x_4 + 0x_5 + 0x_6$
$O : 4x_1 + 4x_2 + 3x_3 = 40x_4 + 3x_5 + 1x_6$
$N : 0x_1 + 2x_2 + 1x_3 = 3x_4 + 2x_5 + 0x_6$
$Mo : 0x_1 + 1x_2 + 0x_3 = 12x_4 + 0x_5 + 0x_6$

As there are 6 variables and 5 equations, we have one free parameter. For convenience, we can take the values corresponding to x_6 to the right side and obtain a square matrix by taking the other values to the left side. The augmented matrix will be:

$$\left[\begin{array}{ccccc|c} 3 & 8 & 1 & -12 & -4 & 2 \\ 1 & 0 & 0 & -1 & -0 & 0 \\ 4 & 4 & 3 & -40 & -3 & 1 \\ 0 & 2 & 1 & -3 & -2 & 0 \\ 0 & 1 & 0 & -12 & -0 & 0 \end{array}\right]$$

Let's try to solve the above system of equations. When implementing the methods discussed in the lectures, you will get values for x_1 to x_5 with respect to the value of x_6. Choose a convenient normalisation to obtain the balancing coefficients with the smallest integer numbers.

(a) Implement an algorithm that solves the above system of equations using your choice of <u>either</u>: Gaussian elimination with back substitution, or Gauss-Jordan elimination, following the procedure described in the lecture.

(b) Now, try to solve the same system using the LU decomposition method, for example by implementing Crout's algorithm, as discussed in the lecture. For this you could make use of parts your code for part (a).

(c) Use the `timeit` module to compare the time required for the above two methods. Which one is more efficient in solving this system of equations? Why?

Solution:

The procedures proposed here are just one possible way to solve the problems, but different approaches can be used.

(a)
1. Write a function to solve the system of equations using Gaussian elimination with back substitution. The function should receive the matrix M given by $[A|b]$ and return an array with the solution x.
2. Within the function, create a for loop that goes through each position of the diagonal elements of the matrix A (let's call this the loop 1). Note that you have to make all the operations with the entire matrix M.
3. Within loop 1, focus on the current diagonal element and on all the elements below it (i.e in the same column but only rows greater than the current one) and search for the

element with the maximum absolute value. Identify the row in which such element is located, and then swap that row (all the columns of that row) by the current one.

4. Still within loop 1, make all the rows below the current diagonal element equal to 0 in the current column. For that you need to add a factor to all the elements of each row below the current position, such that you obtain zero in the current column. After this process you should obtain a matrix $[A'|b']$ with A' an upper triangular matrix. We show below the matrix that you should obtain after this process.

$$\left[\begin{array}{ccccc|c} 4.00 & 4.00 & 3.00 & -40.00 & -3.00 & 1.00 \\ 0.00 & 5.00 & -1.25 & 18.00 & -1.75 & 1.25 \\ 0.00 & 0.00 & 1.50 & -10.20 & -1.30 & -0.50 \\ 0.00 & 0.00 & 0.00 & -13.90 & 0.57 & -0.17 \\ 0.00 & 0.00 & 0.00 & 0.00 & -0.23 & -0.40 \end{array}\right]$$

5. Without loop 1, but still within the function, create an array x, with the same number of elements as b' and create a for loop that goes through each element of the array x, but from the end to the beginning (let's call this the loop 2)
6. Within loop 2, compute the value of x in the current position by dividing the b' in the current position by the diagonal element of A' for the current row.
7. Within loop 2, create another for loop that goes through each row above the current one in the matrix M' (again from the bottom to the top) and substitute the just computed x value in the corresponding columns.
8. The solution of the reduced system is $\{x_1, x_2, x_3, x_4, x_5\} = \{0.083, 1, 1.75, 0.083, 1.75\}x_6$. The solution of the whole system is: $\{x_1, x_2, x_3, x_4, x_5, x_6\} = \{1, 12, 21, 1, 21, 12\}$.
9. We obtain a significant difference in the runtime for the Gaussian elimination with back-substitution and Gauss-Jordan elimination, with the former, Gaussian elimination, being about 6 times faster at 0.00017 sec.

(b) To solve the system of equations using the LU-Decomposition from a library, you can use the command linalg.lu_factor to compute LU and then solve the system using the command linalg.lu_solve. The time required is 0.14 sec, which for us is about 1000 times slower than Gaussian elimination. This, however, is a rather small system of equations, and for larger systems this method becomes better compared to the above two.

(c) See text above for runtimes.

2. **Image Compression via SVD**

In much the same way humans do not need the entirety of an eye's perceptive strength in order to understand most of what is going on, singular value decomposition (SVD) can be used to tell which parts of an image contain the most information. The largest singular values capture the most important information, such as the dominant shapes as encoded by the largest brightness variations. The things we still see even when we squint. The information that is tossed away is instead encoded on the smaller singular values. This is often exploited by modern communication technologies in order to compress images.

Download this, frankly incredible, picture dog_image.jpg. You can load it into `Python` through the following script.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

```
img = mpimg.imread('dog_image.jpg')
grey_img = np.dot(img[..., :3], [0.640, 0.595, 0.155]) # Rec601 hack to turn greyscale
# Do NOT fall into the rabbit hole of that is the
# luminous efficiency function for human eyes.

# In case you need to see the image
plt.imshow(grey_img, cmap="gray")
```

(a) Use `np.linalg.svd`, to calculate the singular value decomposition of the provided picture, that is the U, W, V^T matrices. Reconstitute the original picture via the U, W, V^T matrices.

(b) Compress the picture by truncating the U, W, V^T arrays to their first 5, 10, 50 elements and use the truncated arrays to reconstruct the image. How many singular values do we *really* need to keep the important details of an image?

Solution:

(a) Keeping in mind the way that `numpy` formats its SVD output, we can reconstitute the entire image via:

```
u,s,v = np.linalg.svd(grey_img)
fullimg = np.dot(u[:,:len(s)] * s[:], v[:])
```

Figure 1: and indeed, here he is.

(b) To compress the image, we discard some singular values. This is equivalent to truncating `s`.

```
u,s,v = np.linalg.svd(grey_img)
k = 10
k_image = np.dot(u[:,:k] * s[:k], v[:k])
```

Figure 2: Same picture, only 10 singular values are used

3. **Tridiagonal systems**
Sometimes the matrix for a system of equations has a particular form which makes it possible to solve it more efficiently than e.g. making an LU decomposition. In this exercise we'll see a common example of this: a tridiagonal system. In this case the matrix only has non-zero entries along its diagonal and directly next to it on both sides.

First a reminder for lecture 2: for cubic spline interpolation, given N sample points (x_i, y_i), we saw last week that there is a linear system of equations we have to solve in order to find the three non-trivial polynomial coefficients in each of the $N-1$ intervals. Let's call the polynomial defined on interval $[x_i, x_{i+1}]$ $f_i(x)$. Its value, first and second derivative are then given by:

$$\begin{aligned} f_i(x) &= y_i + b_i(x-x_i) + c_i(x-x_i)^2 + d_i(x-x_i)^3 \\ f_i'(x) &= b_i + 2c_i(x-x_i) + 3d_i(x-x_i)^2 \\ f_i''(x) &= 2c_i + 6d_i(x-x_i). \end{aligned} \tag{1}$$

Note that for $x = x_i$, we just find $f_i(x_i) = y_i$, $f_i'(x_i) = b_i$ and $f_i''(x_i) = 2c_i$. Recall that to get the system of equations to solve, we demand that the values, derivatives and second derivatives on the left and right intervals match at each sample point, i.e. $f_i(x_{i+1}) = f_{i+1}(x_{i+1})$, $f_i'(x_{i+1}) = f_{i+1}'(x_{i+1})$ and $f_i''(x_{i+1}) = f_{i+1}''(x_{i+1})$ for all relevant i. We need two extra demands, which in the case of the natural cubic spline are that $f_0''(x_0) = f_{N-2}''(x_{N-1}) = 0$.

At this point we could write our equations in matrix form $\mathbf{Ax} = \mathbf{b}$ with a $3(N-1)$-dimensional vector of unknowns $\mathbf{x}$ consisting of all b_i, c_i and d_i – but this would be incredibly inefficient! In general, you want to reduce the dimensionality of your problem before you try to solve it numerically. The important thing to realize, in this case, is that (1) all the equations are linear in each of the polynomial coefficients, and (2) that the coefficients of each interval only (directly) depend on those of the neighbouring intervals, e.g. c_4 directly depends on c_3 and c_5 but no other c_i. This means that we might rewrite this system of equations to a tridiagonal one. There are a couple of ways to do so (we could for example write every equation in terms of the second derivative), below we work through just one.

(a) First use $f_i''(x_{i+1}) = f_{i+1}''(x_{i+1})$ to find an expression for d_i. Use the shorthand $\Delta x_i \equiv (x_{i+1} - x_i)$ and don't worry about the boundaries for now. Then substitute this expression for d_i in the equations $f_i(x_{i+1}) = f_{i+1}(x_{i+1})$ and $f_i'(x_{i+1}) = f_{i+1}'(x_{i+1})$. You now have a system of equations only in terms of b_i, b_{i+1}, c_i and c_{i+1}.

(b) Next, use the new equations for $f_i(x_{i+1}) = f_{i+1}(x_{i+1})$ to express b_i in terms of c_i and c_{i+1}. Note that you can also use this same equation to write b_{i+1} in terms of c_{i+1} and c_{i+2}.

(c) Now use the equations of (b) in $f_i'(x_{i+1}) = f_{i+1}'(x_{i+1})$ to find a set of equations in terms of only the coefficients c_i, c_{i+1} and c_{i+2}. Then, substitute $i \to i-1$. Now we are left with just $N-2$ equations, in terms of only c_i and the c-coefficients of the directly neighbouring intervals!

(d) Recall that for the *natural* cubic spline, $c_0 = c_N = 0$, so we have $N-2$ equations with $N-2$ unknowns $\{c_i | i = \overline{1, N-1}\}$. Write the $N-2$ equations you found in (c) in matrix form, with $\mathbf{x} = (c_1\, c_2\, \ldots\, c_{N-1})^{\mathrm{T}}$. You should now have a tridiagonal matrix $\mathbf{A}$!

(e) An LU decomposition of a tridiagonal system only has non-zero entries on the diagonal and one off-diagonal for both the $\mathbf{L}$ and $\mathbf{U}$ matrices. Check that this must indeed be the case, and derive equations for each of the non-zero LU matrix coefficients α_{ij} and β_{ij} in terms of the (non-zero) elements a_{ij} of a tridiagonal matrix $\mathbf{A}$ (remember that $\alpha_{ii} = 1$). You should find that each a_{ij} can be written as the sum of only two terms, only one of which is unknown if you derive the α_{ij} and β_{ij} in order – this means that the LU decomposition can be found very quickly, in only $\mathcal{O}(N)$ operations!

(f) Code up a natural cubic spline interpolation solver using this tridiagonal system and the accompanying LU decomposition. Then test it by taking $N = 4$ and sample points $(0, 2)$, $(1, 0)$, $(2, 1)$ and $(3, 2)$ and solve for the coefficients c_i $(0 < i < N)$. Use substitution to calculate b_i and d_i and plot the four points and your resulting cubic spline.

Solution:

The resulting equation for coefficients c_i is:

$$c_{i-1}\Delta x_{i-1} + 2c_i(\Delta x_{i-1} + \Delta x_i) + c_{i+1}\Delta x_i = 3(p_i - p_{i-1}),$$

where $p_i = (y_{i+1} - y_i)/(x_{i+1} - x_i)$.

The other coefficients can be expressed through c_i as:

$$b_i = p_i - \frac{1}{3}\Delta x_i(c_{i+1} + 2c_i),$$

$$d_i = \frac{1}{3}\frac{c_{i+1} - c_i}{\Delta x_i}.$$

The tridiagonal matrix $\mathbf{A}$ is:

$$\mathbf{A} = \begin{pmatrix} 2(\Delta x_0 + \Delta x_1) & \Delta x_1 & & & \\ \Delta x_1 & 2(\Delta x_1 + \Delta x_2) & \Delta x_2 & & \\ & \Delta x_2 & 2(\Delta x_2 + \Delta x_3) & \Delta x_3 & \\ & & \ddots & \ddots & \ddots \end{pmatrix}.$$

The non-zero values of $\mathbf{L}$ and $\mathbf{U}$ matrices are:

$$\beta_{ii} = a_{ii} - \frac{a_{i(i-1)}a_{(i-1)i}}{\beta_{(i-1)(i-1)}},$$

$$\beta_{i(i+1)} = a_{i(i+1)},$$

$$\alpha_{ii} = 1,$$

$$\alpha_{(i-1)i} = \frac{a_{(i-1)i}}{\beta_{(i-1)(i-1)}}.$$

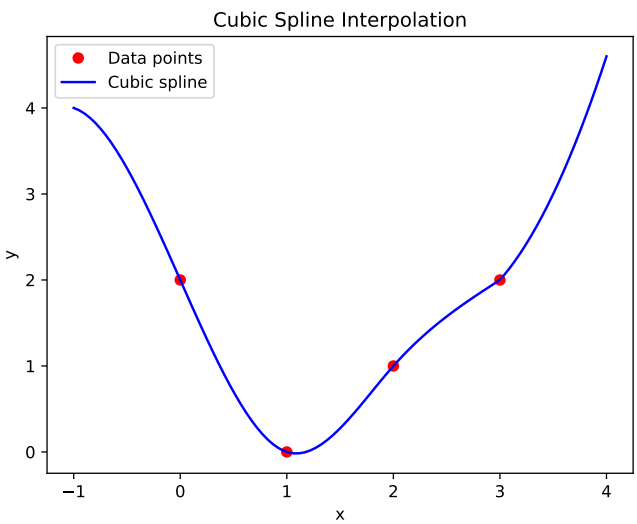

Figure 3: The resulting cubic spline.